

iOS Vibe Coding Prompt Pack

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Version: 1.2

Release date: 12 September 2026

This pack turns vibe coding into an auditable engineering workflow. Each prompt asks an AI coding assistant for a bounded artifact, complete files, and explicit verification. Generated output remains a proposal until the learner reviews the diff and produces red-to-green evidence.

Prerequisites

- Xcode capable of targeting iOS 17 or later, with an available iOS Simulator.
- XcodeGen 2.45.4 or compatible. Run `xcodegen --version`; the Activity `project.yml` files are the editable source of truth and the included `.xcodproj` files are generated outputs.
- Resolve an installed simulator before using any xcodebuild prompt: run `xcrun simctl list devices available`, choose a listed device, and replace `<AVAILABLE\_IOS\_SIMULATOR\_NAME>` in the command. Do not assume a particular model is installed.
- Apple Clang/C++20, CMake 3.24 or later, CTest, Git, and an approved AI coding assistant.
- The BudgetBuddy starter, mock data, and Activity folders. Use no credentials, personal data, proprietary source, signing files, or production analytics data.
- A clean working branch or disposable workspace. Record `git status --short` and the baseline build/test result before asking for code.
- Current Apple mechanics: Observation `@Observable` for iOS 17+ model state, Swift Testing for unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations for the public bridge.

Project context template

Copy this block, replace bracketed values, and include it with a stage prompt.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Non-negotiable output contract

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

Protected interfaces and guardrails

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Iterative red to repair to green workflow

Use this loop for every generated slice:

1. BASELINE - run the smallest relevant verification and save its output.
2. RED - add or strengthen one test that fails for the missing behaviour; retain the failure and exit status.
3. GENERATE - request the smallest causal implementation, using the output contract and protected interfaces.
4. REVIEW - inspect the diff for contract drift, hidden dependencies, unsafe conversions, ownership ambiguity, generic error strings, inaccessible controls, secrets, and unrelated edits.

5. REPAIR - give the assistant the first failing diagnostic plus the exact current file; request one causal patch without weakening tests.
6. GREEN - rerun the focused test, then the full layer suite, then the cross-layer build. Retain commands, exit status, test totals, and artifact paths.
7. RECONCILE - update ADRs and traceability only after code and tests agree.

Expected verification evidence

- Portable layer: clean CMake configure/build, CTest totals, warning status, and optional sanitizer result.
- Bridge layer: iOS build plus exact-cent and stable-error contract tests.
- Swift state layer: Swift Testing results for parsing, fake substitution, state transitions, errors, retry, and persistence coordination.
- UI layer: XCTest/XCUIAutomation results for the visible golden journey, relaunch, and accessibility identifiers.
- Cross-cutting: privacy/security review, documentation link check, traceability completeness, file inventory, and release scorecard.
- For every command, retain the command text, exit status, pass/fail total, first failure when red, and dated evidence path.

End-to-end generation prompts

01 - Discovery and requirements baseline

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Convert the supplied stakeholder notes into REQ-001 through REQ-005. For each, state actor, input, business invariant, observable outcome, negative case, and evidence. Add non-functional constraints for offline use, accessibility, exact-cent fidelity, privacy, and deterministic recovery. Do not design screens yet.

Required file scope:

CREATE docs/requirements.md and docs/acceptance-criteria.md. Do not modify source code.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

```
rg -n 'REQ-00[1-5]|Given|When|Then|Int64|accessib|privacy|recover' docs/requirements.md docs/acceptance-criteria.md must show every requirement, its acceptance case, and the declared constraints.
```

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

02 - Architecture and file plan

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Design the dependency direction and a complete file plan for the smallest BudgetBuddy vertical slice: add an exact-cent expense, reject invalid or over-budget input, persist accepted entries, recover after failure, and expose accessible status. Explain why each dependency points inward.

Required file scope:

CREATE docs/architecture.md and docs/file-plan.md. The plan must include DomainCore CMake/header/source/tests, BudgetBridge.h/.mm, Swift protocol/client/fake, Observation model/view model, SwiftUI app/root/views/controls, persistence/migration, Swift Testing tests, XCTest UI tests, CI, ADRs, traceability, privacy review, and evidence index.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

```
rg -n 'DomainCore|BudgetBridge\.h|BudgetBridge\.mm|@Observable|Swift Testing|XCUIAutomation|PrivacyInfo|traceability' docs/*.md must find every layer and evidence surface.
```

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.

6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

03 - Portable C++ domain models and policy

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Generate a portable C++20 DomainCore with Money, ExpenseId, Category, ExpenseDraft, Decision, DomainError, BudgetPolicy, and an ExpenseRepository port. Money uses std::int64_t cents. Reject non-positive amounts, invalid stored totals, arithmetic overflow, and overspend. Return typed results without exceptions crossing the bridge.

Required file scope:

CREATE DomainCore/include/Money.hpp, Expense.hpp, DomainError.hpp, BudgetPolicy.hpp, ExpenseRepository.hpp and DomainCore/src/BudgetPolicy.cpp. Add no Apple headers.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

```
cmake -S DomainCore -B DomainCore/build && cmake --build DomainCore/build && ctest --test-dir DomainCore/build --output-on-failure; expect configure/build exit 0 and all domain tests passed. rg -n 'Foundation|UIKit|SwiftUI|NSString|NSArray' DomainCore must return no source matches.
```

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

04 - CMake and portable domain tests

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Generate a CMake 3.24+ build and deterministic C++ tests for the domain. Cover accepted exact cents, zero, negative, overspend, overflow boundary, stable identity, repository substitution, and unchanged remaining balance after rejection. Use strict warnings and optional AddressSanitizer flags gated for Clang/GNU.

Required file scope:

CREATE DomainCore/CMakeLists.txt and DomainCore/tests/BudgetPolicyTests.cpp. MODIFY only domain files needed to make the red tests pass.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

Run clean configure, build, and ctest --test-dir DomainCore/build --output-on-failure; expect 100% tests passed and zero warnings. Run the ASan configuration where supported and expect no sanitizer finding.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

05 - Objective-C++ bridge and error mapping

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Generate the sole language-translation adapter. BudgetBridge.h must expose Foundation-compatible immutable DTOs, fixed-width integers, nullability, and NSError**. BudgetBridge.mm must include C++, own the service with std::unique_ptr, convert values safely, map every DomainError to a stable BudgetDomain code and recovery description, and return copied DTOs.

Required file scope:

CREATE BudgetBuddy/Bridge/BudgetBridge.h, BudgetBuddy/Bridge/BudgetBridge.mm, and BudgetBuddyTests/BridgeContractTests.swift. Do not expose C++ in the header.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

Resolve an installed destination with xcrun simctl list devices available, replace <AVAILABLE_IOS_SIMULATOR_NAME>, then run xcodebuild -scheme BudgetBuddy -destination 'platform=iOS Simulator,name=<AVAILABLE_IOS_SIMULATOR_NAME>' build and focused BridgeContractTests. Expect exit 0, exact 10_000/2_500/1_250 cents to return 6_250, and every injected domain error to map to its declared NSError code.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.

5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.

6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

06 - Swift bridge protocol, client, and fake

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Generate the Swift boundary around the imported Objective-C API. Define BridgeDecision and BudgetBridgeError, a BudgetEvaluating protocol, a production BudgetBridgeClient that maps NSError codes without parsing prose, and a configurable FakeBudgetBridge that records inputs and can return a value or throw.

Required file scope:

CREATE BudgetBuddy/Bridge/BudgetEvaluating.swift, BudgetBuddy/Bridge/BudgetBridgeClient.swift, and BudgetBuddyTests/FakeBudgetBridge.swift. Preserve the imported Objective-C method signature.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.

- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

Run the Swift test target. Expect protocol substitution, exact Int64 values, stable error mapping, and captured fake inputs to pass without constructing SwiftUI views.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

07 - Observation model and view model

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Generate the iOS 17+ state layer with import Observation and @Observable. Create BudgetModel for the authoritative snapshot and BudgetViewModel for amount/category input, formatted validation, bridge calls, recoverable status, retry, and loading state. Keep business invariants in C++. Inject

BudgetEvaluating and persistence ports. Make state transitions deterministic and suitable for Swift Testing.

Required file scope:

CREATE BudgetBuddy/Model/BudgetSnapshot.swift, BudgetBuddy/Model/BudgetModel.swift, BudgetBuddy/ViewModel/BudgetViewModel.swift, and BudgetBuddyTests/BudgetViewModelTests.swift.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

Run focused Swift Testing tests. Expect invalid text to preserve input, accepted 12.50 to send 1_250 cents, overspend to expose a recovery message, retry to reuse the draft, and successful save to update remaining cents exactly.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

08 - SwiftUI app, root, views, controls, and accessibility

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Generate a native SwiftUI shell for the BudgetBuddy slice. Provide the @main app, composition root, UINavigationController root, budget summary, expense form, expense list/row, error/retry presentation, and deterministic previews. Use TextField for amount, Picker for the closed Category set, and Button for commit. Add Dynamic Type friendly layout, semantic labels, hints where useful, and stable accessibility identifiers.

Required file scope:

CREATE BudgetBuddy/App/BudgetBuddyApp.swift, BudgetBuddy/App/AppRoot.swift, BudgetBuddy/Views/BudgetScreen.swift, BudgetSummaryView.swift, ExpenseFormView.swift, ExpenseListView.swift, ExpenseRow.swift, and PreviewFixtures.swift.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

Build for an iOS 17+ Simulator. Expect empty, loaded, validation-error, and large-text previews to construct with fakes. UI elements amount, saveExpense, and validationMessage must be discoverable by accessibility automation.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.

3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.

4. VERIFICATION - exact commands in execution order and the expected observation for each.

5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.

6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

09 - Persistence and migration

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Generate a versioned local JSON repository behind the C++ ExpenseRepository port or an explicitly documented adapter port. Use an application-support path supplied by the caller, write a temporary file before atomic replacement, decode schemaVersion, migrate v1 to v2 deterministically, preserve exact cents and IDs, and map corrupt/read/write failures to typed recoverable errors. Never use a real home directory in tests.

Required file scope:

CREATE DomainCore/include/JsonExpenseRepository.hpp, DomainCore/src/JsonExpenseRepository.cpp, DomainCore/tests/JsonExpenseRepositoryTests.cpp, DomainCore/tests/fixtures/snapshot-v1.json, and docs/persistence-schema.md.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.

- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAIL ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

Run clean CTest. Expect v1 migration to v2, save/reload equality, corrupt JSON classification, failed replacement preserving the old snapshot, and all test paths confined to the test temporary directory.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

10 - Swift Testing unit and integration suite

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Generate Swift Testing tests using import Testing, @Test, and #expect for pure Swift and bridge-facing integration behaviour. Cover MoneyParser, view-model state transitions, fake substitution, exact-cent boundary values, mapped bridge errors, retry, and persistence coordination. Keep UI automation out of this target.

Required file scope:

CREATE BudgetBuddyTests/MoneyParserTests.swift, BudgetViewModelTests.swift, BudgetBridgeClientTests.swift, and BudgetPersistenceIntegrationTests.swift. MODIFY production files only for the smallest causal testability seam.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

Resolve an installed destination with xcrun simctl list devices available, replace <AVAILABLE_IOS_SIMULATOR_NAME>, then run xcodebuild test -scheme BudgetBuddy -destination 'platform=iOS Simulator,name=<AVAILABLE_IOS_SIMULATOR_NAME>' -only-testing:BudgetBuddyTests; expect all Swift Testing tests passed, no skipped critical case, and deterministic results on two consecutive runs.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

11 - XCTest UI tests with XCUIAutomation

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Generate XCTest UI tests that launch with deterministic arguments and drive the visible critical path through XCUIApplication. Cover initial summary, entering 12.50, choosing a category, saving, seeing updated remaining amount, invalid input message, over-budget recovery, relaunch restoration, and accessible identifiers. Do not reach into app internals from the UI test.

Required file scope:

CREATE BudgetBuddyUITests/BudgetBuddyUITests.swift and, only if required, a minimal launch-argument seam in BudgetBuddy/App/AppRoot.swift or a test configuration file.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

Resolve an installed destination with `xcrun simctl list devices available`, replace `<AVAILABLE_IOS_SIMULATOR_NAME>`, then run `xcodebuild test -scheme BudgetBuddy -destination 'platform=iOS Simulator,name=<AVAILABLE_IOS_SIMULATOR_NAME>' -only-testing:BudgetBuddyUITests`; expect every queried element to exist and be hittable where relevant, the golden journey to pass, and a second run to produce the same state.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading ``FILE: relative/path`` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

12 - Build-error repair

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use `@Observable` from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Diagnose the pasted first compiler, linker, test, or runtime failure. Explain the causal layer, then produce the smallest repair. Do not silence warnings, weaken assertions, delete negative tests, broaden public APIs, add force unwraps/casts, or replace typed errors with strings. If later diagnostics are cascades, ignore them until the first failure is fixed.

Required file scope:

MODIFY only files directly required by the first diagnostic. Preserve every protected interface unless the evidence proves it is internally inconsistent.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Paste the exact command, first diagnostic, current relevant file, and git diff -- output after this prompt.

Verification that the implementation must enable:

Rerun the smallest failing command, then its complete layer suite, then the cross-layer build. State the expected transition from the exact red diagnostic to a green exit and list retained logs.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

13 - Privacy and security review

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Review the complete BudgetBuddy file inventory and diff for secrets, personal data, unsafe logging, unbounded input, integer overflow, path traversal, insecure file permissions, unexpected networking, analytics/tracking, new SDKs, required-reason API use, privacy manifest impact, and signing/configuration drift. Produce evidence-backed findings with file and line locus. Generate code only for confirmed High or Medium findings.

Required file scope:

CREATE docs/privacy-security-review.md. CREATE or MODIFY PrivacyInfo.xcprivacy only when the reviewed code and Apple requirements justify exact declarations.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

Run secret scanning, dependency inventory, entitlement diff, privacy manifest lint/build validation, focused negative tests, and the full test suite. Expect no untriaged High/Medium finding and no unsupported privacy declaration.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.

5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.

6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

14 - CI build and test workflow

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Generate a CI workflow that builds the portable C++ core first, runs CTest with strict warnings, then builds the iOS scheme and runs Swift Testing plus XCTest UI tests on an available pinned macOS/Xcode runner. Cache only safe build dependencies, upload red/green logs and test results, and never require signing or App Store credentials for validation.

Required file scope:

CREATE .github/workflows/verify.yml and scripts/verify-local.sh. Preserve existing workflow triggers and secrets if supplied.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.

- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

Validate YAML, run scripts/verify-local.sh, and map each CI step to its expected exit status and retained artifact. Expect DomainCore to fail before iOS if the portable layer is red.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

15 - Documentation, ADR, and traceability

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Generate documentation from the verified project inventory. Add a README setup and verification path, ADR-001 for the Objective-C++ boundary, ADR-002 for versioned atomic JSON persistence, public bridge contract documentation for units/ownership/nullability/errors, a C4-style text diagram, and a traceability matrix from REQ-001 through REQ-005 to design, implementation, test, and evidence.

Required file scope:

CREATE README.md, docs/ADR-001-objective-cpp-boundary.md, docs/ADR-002-json-persistence.md, docs/architecture.md, docs/bridge-contract.md, and docs/traceability.csv.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

Run every README command from a clean checkout; check links; compare target dependencies/imports to the diagram; require every traceability row to have a reproducible evidence path and PASS/REVISE status.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

16 - Release evidence

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Assemble a local release-candidate evidence pack without claiming App Store submission. Index the clean CMake/CTest result, iOS build, Swift Testing result, XCTest UI result, accessibility review, persistence recovery, privacy/security review, ADR reconciliation, traceability, file inventory, and known limitations. Distinguish local verification from signing, archive validation, upload, processing, and review.

Required file scope:

CREATE evidence/evidence-index.csv, evidence/release-scorecard.json, evidence/release-notes.md, evidence/deployment-position.md, and evidence/file-inventory.txt.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

Every PASS row must name a command, dated artifact, requirement, owner, and reproducible path. Expect zero blank evidence fields, no secrets, and an explicit HOLD when any mandatory gate is missing.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.

5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.

6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

17 - App Store Connect preparation and gated submission

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Prepare a reproducible App Store Connect handoff without performing a real upload or submission. First reconcile app name, bundle ID, marketing version, build number, deployment target, app icon, usage descriptions, entitlements, encryption declaration, privacy manifest, supported devices, and Release configuration. Then specify the commands and observations for a signed archive, Organizer validation, export, and upload. Create a metadata checklist for description, keywords, URLs, review contact, screenshots, App Privacy, age rating, content rights, pricing and availability. Specify how to wait for build processing, attach the correct build, resolve blockers, submit for review, and read back the resulting state. Mark Apple Developer credentials, signing identities/profiles, real App Store Connect mutations, upload, and submission as TRAINER DEMO / FUTURE CREDENTIALLED GATE. Never request or print secrets and never claim that a gated step ran.

Required file scope:

CREATE release/app-store-preflight.md, release/metadata-checklist.md, release/screenshot-plan.md, release/privacy-age-availability.md, release/archive-export-upload.md, release/submission-readback.md, and release/gate-status.csv. CREATE ExportOptions.plist only as a placeholder-free trainer-supplied template after signing values are known; never invent team, certificate, profile, key, issuer, app, or bundle identifiers.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.

- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAIL ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Use the current Apple help pages linked at the end of this pack. UI labels can change, so record the intent, current observation and Apple-side readback rather than relying on a memorised click label.

Verification that the implementation must enable:

Local gate: regenerate with XcodeGen, build and test on a resolved installed simulator, lint the privacy manifest, verify the 1024x1024 opaque app icon, and inspect the Release build settings. Future credentialled gate: archive, validate, export and upload; confirm processing; set metadata/screenshots/privacy/age rating/availability; attach the intended build; submit; then read back app identity, version, build and review state. Every unexecuted external row must remain HOLD with an owner and next action.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.
6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

18 - Final full-project audit

You are the implementation assistant for BudgetBuddy. Work on one bounded slice and preserve the approved architecture.

Project: BudgetBuddy, a native iOS 17+ expense app.

Regulated outcome: preserve the assessed C++ component-design outcome while using AI to generate and repair code.

Architecture: SwiftUI app and views -> Observation-based model/view model -> Swift bridge protocol/client -> Objective-C-compatible BudgetBridge.h -> BudgetBridge.mm -> portable C++20 DomainCore.

Domain rules: amounts are signed 64-bit integer cents; reject non-positive drafts and overspend; persistence restores accepted entries; errors remain typed and recoverable.

Platform mechanics: use @Observable from Observation for iOS 17+ state, Swift Testing for Swift unit/integration tests, XCTest with XCUIAutomation for UI tests, and Objective-C/C-compatible declarations as the public C++ bridge surface.

Current files: [paste the exact relevant paths and current contents or attach them].

Current failing evidence: [paste the first compiler/test failure, or write NONE for a green baseline].

Requested slice: [one observable behaviour only].

Task:

Audit the final project without making changes first. Compare requirements, file plan, build graph, code, tests, UI identifiers, persistence schema, privacy declarations, ADRs, traceability, CI, and release evidence. Identify invented APIs, placeholder code, truncated files, duplicate policy, C++ leakage into the public header, SwiftUI business rules, Observation misuse, wrong test framework, unstable UI queries, weak error mapping, stale docs, and unsupported release claims. Rank findings by severity and propose the smallest repair sequence.

Required file scope:

READ all project files. CREATE evidence/final-audit.md only after presenting findings. MODIFY no production file until the user selects or approves the repair batch.

Protected interfaces and guardrails:

- Preserve the approved BudgetPolicy::evaluate signature and Int64/std::int64_t cent units unless an explicit requirement authorises a contract change.
- Keep Swift, Objective-C, Foundation, UIKit, and SwiftUI out of DomainCore.
- Keep C++ headers, templates, STL containers, references, and owning pointers out of BudgetBridge.h; C++ includes and RAII ownership belong in .mm.
- Keep the Swift bridge behind a BudgetEvaluating protocol with a deterministic fake.
- Keep business policy out of SwiftUI View.body. Use @Observable for iOS 17+ model state and publish UI mutations on the main actor when concurrency is introduced.
- Swift Testing covers Swift unit/integration behaviour. XCTest/XCUIAutomation covers UI automation.
- Preserve accessibility identifiers `amount`, `saveExpense`, and `validationMessage` once created.
- Do not add credentials, analytics, tracking, network access, third-party SDKs, signing changes, or unrelated formatting.

Verification that the implementation must enable:

After repairs, run portable configure/build/CTest, iOS build, Swift Testing, XCTest UI tests, secret/privacy checks, documentation link checks, traceability completeness, and file inventory comparison. Expect all mandatory gates green and the audit to record exact evidence, not assertions.

Return output in this exact order:

1. ASSUMPTIONS - only assumptions that affect correctness.
2. FILE PLAN - one line per file: CREATE or MODIFY, exact relative path, and purpose.
3. COMPLETE FILES - for every changed file, print a heading `FILE: relative/path` followed by the complete file in one fenced code block.
4. VERIFICATION - exact commands in execution order and the expected observation for each.
5. TRACEABILITY - requirement ID -> file -> test -> retained evidence.

6. REVIEW RISKS - boundary, ownership, integer conversion, error, persistence, accessibility, privacy, and concurrency risks that apply.

No ellipses, TODOs, placeholders, pseudo-code, omitted imports, partial snippets, or invented files. Do not claim that a command ran. If context is missing, stop after ASSUMPTIONS and request the exact missing file or build error.

Completion rule

Stop with HOLD when a protected interface changed without approval, any mandatory verification remains red, a PASS row lacks evidence, privacy/signing scope drift appears, or the project has only local evidence but claims external distribution. Completion means the final full-project audit finds no unresolved mandatory defect and a reviewer can reproduce at least one domain check plus one bridge or UI check from the evidence index.

Official Apple references

- Managing model data in your app: <https://developer.apple.com/documentation/SwiftUI/Managing-model-data-in-your-app>
- Swift Testing: <https://developer.apple.com/documentation/Testing>
- Adding tests to an Xcode project: <https://developer.apple.com/documentation/xcode/adding-tests-to-your-xcode-project>
- Imported C and Objective-C APIs in Swift: <https://developer.apple.com/documentation/swift/imported-c-and-objective-c-apis>
- Running on simulated or physical devices: <https://developer.apple.com/documentation/Xcode/running-your-app-on-simulated-or-physical-devices>
- Upload builds: <https://developer.apple.com/help/app-store-connect/manage-builds/upload-builds>
- Submit an app: <https://developer.apple.com/help/app-store-connect/manage-submissions-to-app-review/submit-an-app>
- Set an app age rating: <https://developer.apple.com/help/app-store-connect/manage-app-information/set-an-app-age-rating>
- Upload app previews and screenshots: <https://developer.apple.com/help/app-store-connect/manage-app-information/upload-app-previews-and-screenshots>
- App information reference: <https://developer.apple.com/help/app-store-connect/reference/app-information/app-information>
- Configure the app icon: <https://developer.apple.com/documentation/xcode/configuring-your-app-icon>
- Privacy manifest files: <https://developer.apple.com/documentation/bundleresources/privacy-manifest-files>
- Required-reason APIs: <https://developer.apple.com/documentation/bundleresources/describing-use-of-required-reason-api>
- Manage App Privacy: <https://developer.apple.com/help/app-store-connect/manage-app-information/manage-app-privacy>